

PORTADA

Técnicas de Programación

Unidad 7 — Desarrollo Web: JavaScript, Fundamentos y Manipulación del DOM

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



TRANSICIÓN

Ya sabemos estructurar y dar estilo. Hoy: comportamiento

En las cuatro sesiones anteriores de esta unidad construimos la **estructura** de una página con HTML (semántica, formularios, tablas) y le dimos **estilo** con CSS3 (selectores, box model, Flexbox, Grid, diseño responsive). Hoy agregamos la tercera pieza: **comportamiento e interactividad**, con JavaScript.

No partimos de cero: ya conocen variables, tipos de datos, condicionales, ciclos y funciones de Java (Unidad 0 y 2), e incluso programación funcional con lambdas y Streams (semana 6). JavaScript reutiliza casi todas esas ideas — la diferencia está en la sintaxis y en el entorno donde vive: **el navegador**.

CONCEPTO

Qué es JavaScript: el lenguaje nativo del navegador

JavaScript es un lenguaje de programación que corre **directamente dentro del navegador**, interpretado y ejecutado por su motor de JavaScript (ej. V8 en Chrome). No requiere un paso de compilación explícito ni una máquina virtual instalada aparte, a diferencia de Java: un programa Java se compila con `javac` a bytecode y se ejecuta sobre la JVM; un script de JavaScript lo recibe el navegador y lo ejecuta tal cual.

JavaScript también puede correr **fuera del navegador** gracias a **Node.js**, un entorno de ejecución que permite usarlo para servidores o scripts — solo lo mencionamos hoy, no lo usaremos en esta sesión.



CONCEPTO

Cómo se vincula JavaScript a una página HTML

Un archivo de JavaScript se conecta a un documento HTML con la etiqueta `<script>` , igual que un archivo CSS se vincula con `<link>` :

```
<script src="app.js" defer></script>
```

Dónde y cómo se carga importa: si el navegador encuentra un `<script>` en el `<head>` sin ningún atributo especial, **detiene el parseo del HTML** hasta descargar y ejecutar ese script — la página se queda "bloqueada" mientras tanto.

El atributo `defer` le dice al navegador: descarga el script en paralelo, pero **ejecútalo solo cuando termine de construir el HTML**. Por eso `defer` (o colocar el `<script>` justo antes de `</body>`) es la práctica recomendada.

CONCEPTO

Variables: `let` y `const`

En Java declaras una variable indicando su tipo: `int edad = 20;` . En JavaScript se usa `let` (variable reassignable) o `const` (no se puede reasignar) — sin declarar el tipo:

```
let edad = 20;
edad = 21;           // válido: let permite reasignar

const nombre = "Ana";
nombre = "Luis";     // Error: no se puede reasignar una const
```

Tanto `let` como `const` tienen **scope de bloque**: solo existen dentro del `{ }` donde se declararon — el mismo comportamiento que ya conocen de Java con variables locales dentro de un `if` o un `for` .

CONTRAEJEMPLO

Por qué `var` ya no se recomienda

Antes de `let` / `const` (ES6, 2015), la única forma de declarar variables era `var`. Su problema: `var` tiene **scope de función**, no de bloque — "se escapa" del `{ }` donde se escribió.

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}  
// Imprime 3, 3, 3 – var "i" es una sola variable  
// compartida fuera del ciclo, no una por iteración  
  
for (let j = 0; j < 3; j++) {  
  setTimeout(() => console.log(j), 0);  
}  
// Imprime 0, 1, 2 – let crea una "j" nueva por iteración
```

Por eso la convención moderna es: usar `const` por defecto, y `let` solo cuando la variable necesita reasignarse. Evitar `var`.

CONCEPTO

Tipos de datos: dinámicos, no declarados

JavaScript es de **tipado dinámico**: una variable no tiene un tipo fijo declarado, el tipo lo determina el valor que contiene en cada momento — y puede cambiar.

Tipo	Ejemplo
number	<code>let n = 42;</code> (enteros y decimales son el mismo tipo)
string	<code>let s = "hola";</code>
boolean	<code>let b = true;</code>
undefined	variable declarada, sin valor asignado
null	ausencia de valor, asignada intencionalmente
object	<code>let p = { nombre: "Ana" };</code>
array	<code>let lista = [1, 2, 3];</code> (es un <code>object</code> especial)

CONTRAEJEMPLO

La flexibilidad del tipado dinámico tiene un precio

En Java, sumar un `int` con un `String` sin conversión explícita es un **error de compilación** — el compilador lo detecta antes de ejecutar el programa. En JavaScript, la misma operación se ejecuta sin quejarse, con un resultado que puede no ser el esperado:

```
let total = "5" + 3;    // "53"   (concatena, no suma)
let resta = "5" - 3;    // 2      (aquí sí convierte a número)
let x;
console.log(x + 1);     // NaN    (undefined + número)
```

Ese es el precio de no declarar tipos: **flexibilidad** para escribir rápido, a cambio de errores que solo aparecen **en tiempo de ejecución** — y que en Java el compilador habría detenido antes de siquiera correr el programa.

CONCEPTO

Operadores y estructuras de control: sintaxis casi idéntica

Aquí no hay mucho que explicar de cero: `if / else` , `for` y `while` en JavaScript se escriben prácticamente igual que en Java.

```
let nota = 4.2;
if (nota >= 3.0) {
  console.log("Aprobado");
} else {
  console.log("Reprobado");
}

for (let i = 0; i < 5; i++) {
  console.log(i);
}

let intentos = 0;
while (intentos < 3) {
  intentos++;
}
```

Los operadores de comparación (`==` , `>` , `&&` , `||`) y aritméticos son los mismos. Un detalle: usa siempre `===` (igualdad estricta, sin conversión de tipo) en vez de `==` — evita sorpresas de coerción como las de la diapositiva anterior.

CONCEPTO

Funciones: declaración clásica y *arrow functions*

JavaScript tiene dos formas principales de escribir funciones:

```
// Declaración clásica
function sumar(a, b) {
  return a + b;
}

// Arrow function (función flecha)
const sumar = (a, b) => a + b;
```

La forma clásica se prefiere para funciones con nombre que se reutilizan en varias partes del archivo. Las *arrow functions* se prefieren para funciones cortas y anónimas, sobre todo como argumento de otra función — el mismo rol que cumplen las **lambdas de Java** que ya usaron con Streams en la semana 6.

TRANSICIÓN

Del lenguaje a la página: JavaScript y el DOM

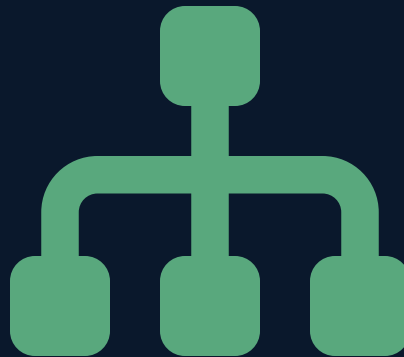
En la primera sesión de esta unidad mencionamos, sin entrar en detalle, que el navegador tiene un **motor de renderizado** que convierte el HTML en lo que se ve en pantalla. Ahora vamos a detallar exactamente esa pieza — el **DOM** — y cómo JavaScript la usa para modificar la página después de que ya se cargó.

CONCEPTO

El DOM: la página convertida en un árbol de objetos

El **DOM** (*Document Object Model*) es la representación en memoria de la página que el motor de renderizado construye a partir del HTML: cada etiqueta se convierte en un **nodo** de un árbol de objetos, con relaciones de padre/hijo/hermano.

Esa estructura en memoria es exactamente lo que JavaScript puede **leer y modificar en tiempo real**, sin recargar la página: agregar un elemento, cambiar un texto, quitar una clase CSS. Cada cambio en el DOM hace que el navegador vuelva a pintar la parte afectada de la pantalla.



CONCEPTO

Seleccionar elementos: `querySelector`

Para modificar un nodo del DOM primero hay que encontrarlo. Los métodos más usados aceptan cualquier selector CSS (los mismos que ya conocen de la unidad de CSS):

```
// Devuelve el PRIMER elemento que coincide
const boton = document.querySelector(".btn-enviar");

// Devuelve TODOS los elementos que coinciden (NodeList)
const items = document.querySelectorAll("li.tarea");

items.forEach(item => console.log(item.textContent));
```



CONCEPTO

Modificar el contenido: `textContent` e `innerHTML`

```
const parrafo = document.querySelector("#mensaje");

parrafo.textContent = "Guardado con éxito"; // solo texto plano
parrafo.innerHTML = "<strong>Guardado</strong>"; // interpreta HTML
```

`textContent` reemplaza el texto sin interpretarlo como HTML — es la opción segura por defecto. `innerHTML` sí interpreta etiquetas, lo que es útil para insertar contenido con formato, pero es **riesgoso si el contenido viene de un usuario no confiable**: alguien podría inyectar HTML o scripts no deseados. Regla práctica: usa `textContent` salvo que realmente necesites insertar HTML, y nunca con datos externos sin validar.

CONCEPTO

Cambiar apariencia: `classList`

Para cambiar el estilo de un elemento no se edita CSS "a mano" desde JS — se le agrega o quita una **clase** que ya tiene sus reglas definidas en la hoja de estilos, con `classList` :

```
const tarjeta = document.querySelector(".card");

tarjeta.classList.add("activa");    // agrega la clase
tarjeta.classList.remove("activa"); // la quita
tarjeta.classList.toggle("activa"); // la agrega si no está, la quita si está
```

Este patrón — clase agregada/quitada por JS, estilo definido en CSS — mantiene separadas las responsabilidades: JS decide **cuándo** cambia el estado, CSS decide **cómo se ve** ese estado.

CONCEPTO

Eventos: reaccionar a la interacción del usuario

`addEventListener` registra una función que se ejecuta cuando ocurre un evento sobre un elemento — un clic, una tecla, un envío de formulario:

```
const boton = document.querySelector("#btn-saludar");

boton.addEventListener("click", () => {
  const parrafo = document.querySelector("#saludo");
  parrafo.textContent = "¡Hola!";
  parrafo.classList.add("resaltado");
});
```

El primer argumento es el nombre del evento (`"click"` , `"submit"` , `"keydown"` ...), el segundo es la función que responde a ese evento — típicamente una arrow function.



CONCEPTO

El evento `submit` y `preventDefault()`

En la sesión de HTML de esta unidad construimos formularios con `<form>` . Por defecto, al enviarlo el navegador **recarga la página** — algo que casi nunca queremos cuando validamos o procesamos el formulario con JavaScript:

```
const formulario = document.querySelector("#form-login");

formulario.addEventListener("submit", (event) => {
  event.preventDefault(); // evita la recarga por defecto
  const usuario = document.querySelector("#usuario").value;
  console.log("Enviando:", usuario);
});
```

`event.preventDefault()` cancela esa acción por defecto del navegador, dejando que sea el propio JavaScript quien decida qué hacer con los datos del formulario.

INTERACCIÓN

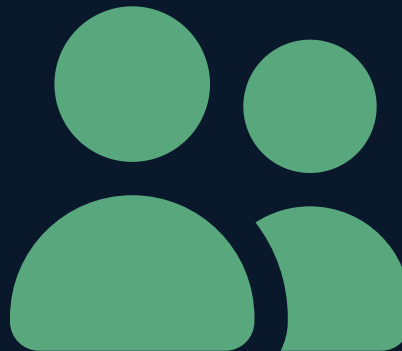
Manos al teclado: click, texto y clase

En parejas: partiendo de este HTML, escriban el JavaScript que, al hacer clic en el botón, cambie el texto del párrafo y le agregue una clase CSS.

```
<button id="btn-confirmar">Confirmar</button>  
<p id="estado">Pendiente...</p>
```

Pistas: `document.querySelector` , `addEventListener("click", ...)` , `textContent` y `classList.add` .

8 minutos · compartan su solución con otra pareja



SÍNTESIS

Lo que hay que llevarse de hoy

1. JavaScript corre **nativamente en el navegador**, sin compilación previa ni JVM; `defer` evita que bloquee el renderizado.
2. Usa `const` por defecto y `let` si necesitas reasignar; evita `var` por su scope de función.
3. El tipado es **dinámico**: flexible, pero sin la red de seguridad del compilador de Java.
4. Operadores, `if`, `for` y `while` son casi idénticos a Java; las funciones tienen forma clásica y *arrow*.
5. El **DOM** es la página en memoria; JS la lee y modifica en tiempo real con `querySelector`, `textContent` / `innerHTML` y `classList`.
6. `addEventListener` conecta el código con la interacción del usuario; `preventDefault()` controla el comportamiento por defecto del navegador.



CIERRE

Antes de la próxima sesión

- Practicar con el ejercicio de la interacción de hoy: agregar un segundo botón que quite la clase en vez de agregarla.
- Repasar `map` , `filter` y `reduce` de los Streams de Java (semana 6) — la próxima sesión tienen su equivalente directo en JavaScript.

Próxima sesión: Funciones de orden superior (`map`, `filter`, `reduce`) y consumo de una API REST con `fetch`.

